



Introduction to Programming

JavaScript Fundamentals

Chapter Overview

- 01** Introduction to JavaScript
- 02** Variables and Data Types
- 03** JavaScript Arrays
- 04** Operators and Control Structures



Introduction to JavaScript

Section 1



What is JavaScript?

A Brief History

- Created by Brendan Eich at Netscape in 1995
- Originally designed to add interactivity to web pages

Today's JavaScript

- The most widely used programming language in the world
- Runs in every web browser — no installation needed
- Also runs on servers, mobile devices, and IoT hardware



The Web Triad: HTML, CSS, and JavaScript

The Web Development Triad



Together they form the three pillars of every modern web page

Client-Side vs Server-Side JavaScript

Client-Side (Browser)

- Runs directly in the user's web browser
- Manipulates the page content (DOM), handles user events
- Examples: form validation, animations, dynamic menus

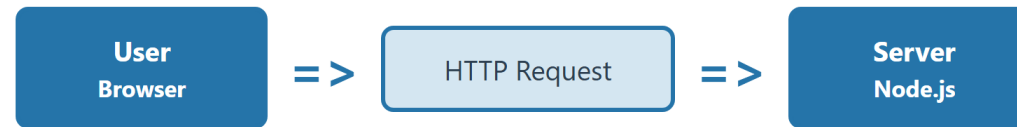
Server-Side (Node.js)

- Runs on a server using the Node.js runtime
- Handles databases, file I/O, and API endpoints
- Same language on both sides — full-stack JavaScript



Client-Server Architecture

Client-Side vs. Server-Side JavaScript



Client-Side JS

- > Runs in the browser
- > Reacts to user events (click, type)
- > Manipulates the DOM
- > Validates forms instantly
- > No server round-trip needed

Server-Side JS (Node.js)

- > Runs on the server
- > Handles API requests
- > Accesses databases
- > File system operations
- > Same language, different runtime

Adding JavaScript to a Web Page

Embedded Scripts

- Place code inside `<script>` tags in the HTML file
- Good for small, page-specific behaviors

External Scripts

- Write code in a separate .js file and link with `<script src>`
- Keeps HTML clean, enables caching and reuse across pages

Best Practice

- Use external scripts for all production code



Embedded vs External Scripts

```
<!-- Embedded script -->
<script>
  alert("Hello from embedded script!");
</script>

<!-- External script -->
<script src="app.js"></script>

// app.js
console.log("Hello from external script!");
```

Dialog Boxes and Node.js

Built-in Dialog Boxes

- `alert()` — displays a message popup to the user
- `confirm()` — asks a yes/no question, returns true or false
- `prompt()` — asks the user to type a value, returns a string

Running JavaScript with Node.js

- Install Node.js from nodejs.org — includes the npm package manager
- Run files from the terminal: `node app.js`
- Interactive REPL: type `node` with no arguments to experiment



Dialog Boxes in Action

```
// Alert – simple notification
alert("Welcome to JavaScript!");

// Confirm – returns true or false
let proceed = confirm("Delete this file?");
console.log(proceed); // true or false

// Prompt – returns user input as a string
let name = prompt("What is your name?");
console.log("Hello, " + name + "!");
```



Variables and Data Types

Section 2



JavaScript Data Types

JavaScript Data Types

Primitive Types (Immutable)

Number

String

Boolean

null

undefined

Symbol

BigInt

42 "hello" true null undefined

Reference Types (Mutable)

Object

Array

Function

Date

RegExp

{name: "Jo"} [1, 2, 3] () => {} new Date()

typeof tells you the type: typeof 42 === "number" | typeof "hi" === "string"



Common Primitive Data Types

String

- Text enclosed in quotes: 'hello' or "hello" or `hello`

Number

- Integers and decimals: 42, 3.14, -7 (no separate int/float)

Boolean

- Logical values: true or false

Undefined & Null

- undefined — variable declared but no value assigned
- null — intentionally empty or absent value



Declaring Variables: var, let, and const

var vs. let vs. const

	var (legacy)	let	const (preferred)
Scope	Function	Block {}	Block {}
Reassign?	Yes	Yes	No
Redeclare?	Yes	No	No
Hoisted?	Yes (undefined)	TDZ error	TDZ error
Best for	Avoid	Counters, flags	Everything else

TDZ = Temporal Dead Zone — accessing before declaration throws ReferenceError



Declaring Variables

let — Block-Scoped Variable

- Can be reassigned, only accessible inside its block { }
- Preferred for values that change over time

const — Block-Scoped Constant

- Cannot be reassigned after declaration
- Use by default — switch to let only when needed

var — Function-Scoped (Legacy)

- Older keyword, hoisted to the top of the function
- Avoid in modern code — use let or const instead

Variable Declaration Examples

```
// const – cannot be reassigned
const PI = 3.14159;
const APP_NAME = "MyApp";

// let – can be reassigned
let score = 0;
score = 10;           // OK
score = score + 5;    // score is now 15

// Dynamic typing – type can change
let value = 42;        // number
value = "hello";       // now a string
typeof value;         // 'string'
```

Strings and Escape Sequences

Creating Strings

- Single quotes: 'Hello' Double quotes: "Hello"
- Template literals with backticks: `Hello, \${name}!`

Common Escape Sequences

- `\n` — newline, `\t` — tab, `\\` — backslash
- `\'` — single quote inside single-quoted string

Useful String Properties

- `.length` returns the number of characters
- `.toUpperCase()`, `.toLowerCase()`, `.trim()`, `.includes()`



Working with Strings

```
// Template literals (backticks)
const name = "Alice";
const greeting = `Hello, ${name}! You are ${2024 - 1990} years old.`;

// String methods
const text = "  JavaScript is Fun!  ";
text.length;           // 23
text.trim();           // "JavaScript is Fun!"
text.toUpperCase();    // "  JAVASCRIPT IS FUN!  "
text.includes("Fun");  // true
"Hello".repeat(3);     // "HelloHelloHello"
```



Numbers and the Number Object

JavaScript Number Basics

- All numbers are 64-bit floating point (IEEE 754)
- No separate integer type — 5 and 5.0 are the same

Special Values

- Infinity, -Infinity — result of dividing by zero
- NaN (Not a Number) — result of invalid math operations

Useful Functions

- parseInt() and parseFloat() — convert strings to numbers
- Number.isInteger(), Number.isNaN(), .toFixed()



Objects and Built-in Objects

What is an Object?

- A collection of key-value pairs (properties and methods)
- Almost everything in JavaScript is an object or behaves like one

The Date Object

- `new Date()` creates a date/time object for the current moment
- Methods: `getFullYear()`, `getMonth()`, `getDate()`, `toLocaleDateString()`

The Math Object

- `Math.round()`, `Math.floor()`, `Math.ceil()`, `Math.random()`
- `Math.max()`, `Math.min()`, `Math.PI`



Anatomy of a JavaScript Object

Anatomy of a JavaScript Object

```
const account = {}
```

Property	Value	Type
name	"John Doe"	String
balance	5000	Number
active	true	Boolean
deposit	function()	Method

```
account.name  
=> "John Doe"
```

```
account["balance"]  
=> 5000
```

```
account.deposit(100)  
=> balance is now 5100
```

Objects in Practice

```
// Creating an object
const student = {
  name: "Alice",
  age: 22,
  enrolled: true,
  greet() { return `Hi, I am ${this.name}`; }
};

console.log(student.name);    // "Alice"
console.log(student["age"]);  // 22

// Date object
const now = new Date();
console.log(now.getFullYear()); // 2026
```

JavaScript Arrays

Section 3



Introduction to Arrays

What is an Array?

- An ordered list of values stored in a single variable
- Elements are accessed by their index (starting at 0)

Creating Arrays

- Array literal: `const colors = ['red', 'green', 'blue']`
- Constructor: `const nums = new Array(1, 2, 3)`

Key Characteristics

- Arrays can hold mixed types: `[42, 'hello', true, null]`
- Length is dynamic — arrays grow and shrink automatically



Array Operations at a Glance

Array Operations Visualized

```
const fruits = ["Apple", "Banana", "Cherry"];
```

[0] Apple [1] Banana [2] Cherry

`fruits.push("Date") => adds to END`

Apple Banana Cherry Date <=

`fruits.pop() => removes from END`

Apple Banana Cherry X =>

`fruits.splice(1, 1, "Blueberry", "Coconut") => remove at [1], insert 2`

Apple Blueberry Coconut Cherry



Creating and Accessing Arrays

```
// Creating arrays
const fruits = ["apple", "banana", "cherry"];

// Accessing elements by index (0-based)
fruits[0];           // "apple"
fruits[2];           // "cherry"
fruits.length;       // 3

// Modifying elements
fruits[1] = "blueberry";
console.log(fruits); // ["apple", "blueberry", "cherry"]

// Accessing out-of-bounds index
fruits[10];          // undefined (no error)
```

Adding and Removing Elements

Adding Elements

- `push(item)` — adds to the end of the array
- `unshift(item)` — adds to the beginning of the array
- `splice(index, 0, item)` — inserts at a specific position

Removing Elements

- `pop()` — removes and returns the last element
- `shift()` — removes and returns the first element
- `splice(index, count)` — removes count items at position



Modifying Arrays

```
const tasks = ["eat", "code", "sleep"];

// Adding
tasks.push("repeat");      // end: ["eat","code","sleep","repeat"]
tasks.unshift("wake up");  // start: ["wake up","eat","code",...]

// Removing
tasks.pop();               // removes "repeat"
tasks.shift();             // removes "wake up"

// Insert at index 1
tasks.splice(1, 0, "learn"); // ["eat","learn","code","sleep"]

// Delete 1 item at index 2
tasks.splice(2, 1);         // ["eat","learn","sleep"]
```

Searching and Converting Arrays

Finding Elements

- `indexOf(value)` — returns position, or -1 if not found
- `includes(value)` — returns true or false
- `find(fn)` — returns the first element matching a condition

Converting Arrays

- `toString()` — comma-separated string: '1,2,3'
- `join(separator)` — custom separator: '1 - 2 - 3'

Sorting

- `sort()` — alphabetical by default (even for numbers!)
- `sort((a, b) => a - b)` — numeric ascending sort



Essential Array Methods

Array Methods Cheat Sheet

Add / Remove

```
.push(item)    => add to end  
.pop()         => remove from end  
.unshift(item) => add to start  
.shift()       => remove from start  
.splice(i,n,...) => insert/remove at i
```

Search / Test

```
.indexOf(item) => first index (-1)  
.includes(item) => true / false  
.find(fn)      => first match  
.findIndex(fn) => index of match  
.every(fn)     => all pass test?
```

Transform

```
.map(fn)       => new transformed array  
.filter(fn)    => new filtered array  
.reduce(fn, init) => single value  
.flat()        => flatten nested  
.slice(s,e)    => copy sub-array
```

Order / Convert

```
.sort()        => sort in place  
.sort((a,b)=>a-b) => numeric sort  
.reverse()     => reverse in place  
.join(sep)     => string  
.toString()    => comma string
```

Methods returning "new" arrays do not mutate the original | .sort() and .reverse() mutate in place



Traversing Arrays

for Loop

- Classic index-based iteration with full control

for...of Loop

- Modern syntax — iterates over values directly

forEach() Method

- Calls a function for each element — clean and expressive

filter() and map()

- filter() returns a new array with elements that pass a test
- map() returns a new array with each element transformed



Iterating Over Arrays

```
const scores = [85, 92, 78, 95, 88];

// for...of – iterate values
for (const score of scores) {
  console.log(score);
}

// filter – keep scores above 90
const high = scores.filter(s => s > 90); // [92, 95]

// map – add 5 bonus points to each
const curved = scores.map(s => s + 5);    // [90, 97, 83, 100, 93]

// forEach – log each with index
scores.forEach((s, i) => console.log(`#${i+1}: ${s}`));
```

Operators and Control Structures

Section 4



JavaScript Operators Overview

JavaScript Operators at a Glance

Arithmetic

```
5 + 3 => 8
10 - 4 => 6
3 * 7 => 21
15 / 4 => 3.75
15 % 4 => 3 (remainder)
2 ** 3 => 8 (exponent)
```

Comparison

```
5 == "5"    => true  loose
5 === "5"   => false strict
5 !== "5"   => true
10 > 5       => true
3 >= 3       => true
Always prefer === over ==
```

Assignment

```
x = 10    assign
x += 5    x = x + 5
x -= 3    x = x - 3
x *= 2    x = x * 2
x++       x = x + 1
x--       x = x - 1
```

Logical

```
true && false => false AND
true || false => true  OR
!true         => false NOT
```

Ternary:

```
(x > 5) ? "big" : "small"
```



Arithmetic and String Operators

Arithmetic Operators

- + addition, - subtraction, * multiplication
- / division, % modulus (remainder), ** exponentiation
- ++ increment, -- decrement

String Operators

- + concatenates strings: 'Hello' + ' World' → 'Hello World'
- += appends: greeting += '!' adds '!' to the end

Comparison Operators

- == loose equality, === strict equality (preferred)
- != not equal, !== strict not equal, < > <= >=



Logical Operators and Type Coercion

Logical Operators

- && (AND) — true only when both sides are true
- || (OR) — true when at least one side is true
- ! (NOT) — flips true to false and vice versa

Type Coercion Pitfalls

- == converts types before comparing: '5' == 5 is true
- === does NOT convert: '5' === 5 is false
- Always use === to avoid unexpected behavior



Operators in Action

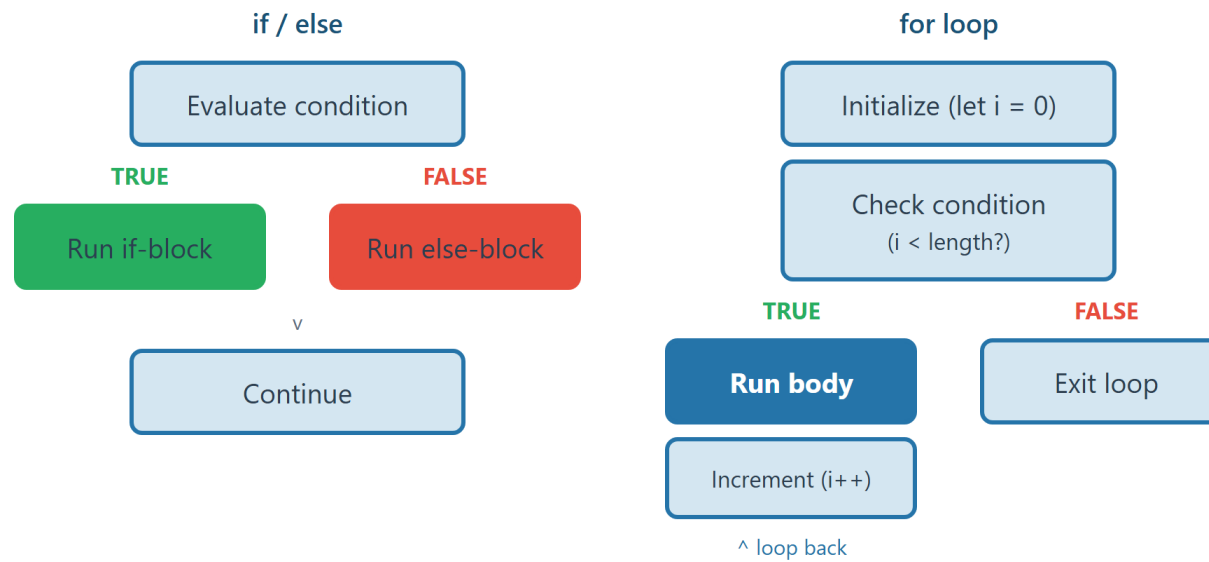
```
// Arithmetic
let total = 10 + 5;      // 15
let remainder = 17 % 5; // 2
let power = 2 ** 8;      // 256

// String concatenation vs template literal
let msg = "Score: " + total;      // "Score: 15"
let msg2 = `Score: ${total}`;     // Same result, cleaner

// Strict vs loose equality
console.log("5" == 5);    // true  (coercion)
console.log("5" === 5);   // false (no coercion)
console.log(null == undefined); // true
console.log(null === undefined); // false
```

Control Flow at a Glance

Control Flow Patterns



Conditional Statements: if / else

if Statement

- Executes a block of code only when the condition is true
- else — runs when the if condition is false
- else if — chains multiple conditions together

Truthy and Falsy Values

- Falsy: false, 0, "", null, undefined, NaN
- Everything else is truthy — including '0' and empty arrays



if/else and switch Statements

```
// if / else if / else
const score = 85;
if (score >= 90) {
  console.log("A");
} else if (score >= 80) {
  console.log("B");    // ← this runs
} else {
  console.log("C or below");
}

// switch statement
const day = "Monday";
switch (day) {
  case "Monday": console.log("Start of week"); break;
  case "Friday": console.log("Almost weekend"); break;
  default: console.log("Regular day");
}
```

Loops: for and for...in

for Loop

- Three parts: initialization; condition; update
- Runs a block repeatedly while the condition is true
- Most common loop for numeric iteration and arrays

for...in Loop

- Iterates over the property names (keys) of an object
- Useful for inspecting object properties
- Avoid for arrays — use for...of or forEach instead

for and for...in Loops

```
// Classic for loop
for (let i = 0; i < 5; i++) {
  console.log("Iteration:", i); // 0, 1, 2, 3, 4
}

// Looping over an array
const colors = ["red", "green", "blue"];
for (let i = 0; i < colors.length; i++) {
  console.log(colors[i]);
}

// for...in – iterate object keys
const car = { make: "Toyota", year: 2024 };
for (const key in car) {
  console.log(`${key}: ${car[key]}`);
}
```

Loops: while and do...while

while Loop

- Checks the condition BEFORE each iteration
- May never execute if the condition starts as false
- Best when you don't know how many iterations you need

do...while Loop

- Checks the condition AFTER each iteration
- Always executes at least once
- Useful for menu prompts and input validation



while and do...while Loops

```
// while – count down
let count = 5;
while (count > 0) {
  console.log("T-minus", count);
  count--;
}
console.log("Liftoff!");

// do...while – ask until valid
let input;
do {
  input = prompt("Enter a number > 10:");
} while (Number(input) <= 10);
console.log("You entered:", input);
```

Key Takeaways

Key Takeaways

1. JavaScript is the language of the web — runs in browsers and on servers via Node.js
2. Use const by default, let when reassignment is needed, avoid var
3. JavaScript has dynamic typing — variables can change type at runtime
4. Arrays are ordered collections with powerful built-in methods like map, filter, and sort
5. Always use === for comparison to avoid type coercion surprises
6. Master control flow (if/else, loops) to write logic that responds to any condition

